

Unified Portal — Cross-Platform UX & Information Architecture Specification

Status: Developer handoff, v1 **Scope:** Consolidation of three existing self-contained platforms into one product with a single backend and one coherent web layer for clients and admins. **What this document is:** the architecture, the integrity model, the shared page template, the per-platform specifications, the decision log, and the recommended build sequence. It is the contract the rework builds against. **What this document is not:** screen-level visual layouts, the actual questionnaire/question sets, the database schema, or the AI engine internals. Those are deliberately downstream of this and noted in Section 11.

1. The product in one paragraph

This is one product made of three independent assessment platforms that share a common spine. Platform 1 identifies an organization's technical debt by finding overlapping, redundant, and shadow-IT tooling. Platform 2 assesses an organization's compliance posture against a chosen zero-trust / cybersecurity framework. Platform 3 maps an organization's actual exploitable attack surface against the MITRE ATT&CK matrix and identifies coverage gaps. The three platforms run independently, but every one of them depends on the same artifact — the client's software/capability list — and that shared dependency, plus a shared repository, shared identity, shared search, and a shared audit log, is what makes this one product rather than three. The strategic payoff (Section 9) is the loop between them: the gaps Platform 3 finds can be funded by the waste Platform 1 finds, with Platform 2's roadmap as the compliance framing. The platforms are independent as workflows but compose into a single narrative.

2. Non-negotiable architectural principles

These are the spine of the rework. If schedule pressure forces cuts, cuts come from scope, never from these. A version of this product that drops the integrity model is not a cheaper version of this product — it is a different and untrustworthy product.

One product, one backend. The JSON and Python consolidation is the developer's implementation call. The UX requirement it must serve: shared identity, one repository, consistent screens. A merged backend enables a unified web layer; it does not guarantee one. Three disjointed UIs on one backend is a failure of this spec.

The repository is the spine, not a feature. Every platform reads from and writes to one shared document repository. Platforms are workflows over shared documents, not silos that own files.

The client tier sits above projects. The capability list is a per-client asset, versioned, that lives above individual projects and engagements. Every project across all three platforms, for a given client, references that client's capability list rather than holding its own copy. Versioning is mandatory, not optional, because an evolving list reused across projects will change underneath projects that already consumed an earlier state.

Origin is assigned once, at creation, and is immutable. Every artifact has an origin that never changes. Humans write only to the human lane. AI processing steps write only to the AI lane. A file never moves between lanes — it is copied forward, and the original stays in place with its provenance intact. Folder structure is how you organize; origin immutability is how you guarantee integrity. If an AI artifact can be edited in place until it is indistinguishable from a human document, the folder structure is decoration.

Promotion elevates reuse status; it never rewrites origin. A human reviewing and approving an AI artifact makes it reusable downstream. It does not make it human source data. A promoted AI artifact is permanently labeled "AI-origin, human-approved," not relabeled as source. Promotion must never read in the UI as laundering.

Integrity is enforced at the reuse moment, not at storage. The risk this product must prevent is AI output being consumed later as if it were source data. That mistake happens at selection time, not at save time. Therefore the integrity UX lives in the artifact picker, not in the folder structure. The picker is where origin must be the loudest element on the screen.

One reusable page template, instantiated per platform. The three platforms are parallel and independent, which is a gift: you build one page-archetype set and instantiate it three times. Only two things differ per platform — the stage-page verbs (review / analyze / generate) and the type-specific metadata fields. Everything else is built once.

Versioning travels with every reuse. Whenever any platform consumes the capability list (or any shared artifact), it records which version, as of which date. "Platform 2 used the capability list" is meaningless without the version stamp.

3. Environment & compliance constraints

GCC High. This is a front-end constraint, not just hosting. No assets, fonts, icons, or scripts from arbitrary public CDNs — self-host everything. Treat any third-party SaaS component (analytics, session replay, embedded chat, font services) as unauthorized until proven otherwise. Assume authentication runs through the GCC High Entra ID tenant: design for SSO and conditional access, and plan for the possibility of CAC/PIV. Design the front end as self-contained from day one; retrofitting this later is expensive.

Accessibility. No ADA mandate has been stated. Recommendation, on the record: build to WCAG 2.1 AA as a baseline design discipline regardless. For federal/government clients, Section

508 (which points at WCAG) is usually legally required, and "not required yet" tends to become "required during an audit." Most of WCAG AA (keyboard operability, contrast, visible focus, clear labels) is just good UX and is cheap to build in and expensive to bolt on. USWDS (U.S. Web Design System) is optional, self-hostable (GCC-High-compatible), accessible out of the box, and familiar to government users; not mandated here, but it removes the need to build a design system from scratch.

Client-supplied content is data, never instructions. Files and answers submitted by clients (especially via link-opened intake surfaces) are treated strictly as data. Nothing a client uploads or types influences processing logic or system behavior.

4. The origin & integrity model

This is the heart of the product. Every artifact in the repository has exactly one origin, set at creation, immutable thereafter.

Origin types:

- **Human-input (source).** Uploaded or entered by a person — client or admin. The source of truth. Read-only to AI processing. Carries actor (client vs admin) and stage metadata.
- **AI-generated.** Produced by an AI processing step — extractions, overlap analysis, posture analysis, gap analysis, conversational answers. Always labeled as AI origin. Never human-writable.
- **Human-authored, AI-informed.** A human-authored synthesis that incorporates AI findings — Platform 1's admin-final list, Platform 2's roadmap. This is a distinct third origin: not "human source" (it derives from AI analysis) and not "AI output" (a human made the decisions). It must be labeled as its own type so downstream consumers know which of the lists they are holding.
- **Promoted.** Status, not a separate origin. An AI-generated artifact a human has explicitly reviewed and approved for downstream reuse. Origin remains "AI-generated"; reuse status becomes "approved." The lineage marker is indelible.

Platform 2 adds input trust tiers (these are sub-classifications of human-input, all auditable separately):

- **Client-asserted** — client claimed it, in a client-only session, no admin present.
- **Admin-assisted** — admin was in the session; client answered but the admin may have shaped it. Conservative default whenever an admin is present.
- **Admin-entered-on-behalf** — admin entered it because the client was unavailable.
- **Client-provided evidence** — files the client uploaded as proof. Human-input source, actor=client.

The reuse-moment rule. Using an AI-origin artifact as a processing input requires an explicit, deliberate acknowledgment ("I understand this is AI-generated, not source data") presented at the moment of selection in the picker. This single gate, placed exactly where the risk occurs, is the primary defense for the stated integrity goal. AI-origin items in any picker must be visually unmistakable and visually distinct from human-input items.

The promotion rule. Promotion is always an explicit human action, never automatic. Only AI-origin artifacts can be promoted, only by an authorized human, only with a recorded reason. Promotion writes an audit entry and a status change; it never alters origin.

5. Shared conceptual data model (entities, not schema)

The schema is the developer's; these are the entities the UX assumes exist and the relationships it depends on.

- **Client / Organization** — top tier. Owns the capability list and all projects.
 - **Capability list** — per-client, versioned. The shared spine asset. Has many versions over time; every version is retained; every consumption references a specific version.
 - **Project / Engagement** — belongs to a client, belongs to exactly one platform. References (does not copy) the client's capability list, by version.
 - **Artifact** — any document or structured output. Has: immutable origin, lineage (what created it, from which inputs, in which project/stage, by whom or by what process), version history, downstream-usage references, and (when applicable) the capability-list version it was based on.
 - **Lane** — the partition of the repository an artifact lives in, determined by origin. Per-project, but the human / AI / promoted lane structure repeats identically inside every project.
 - **Audit entry** — append-only record of every state change, promotion, attribution, and reuse.
-

6. The shared page template

Read every page below as "this page, in all three platforms." Build once, instantiate three times. Differences between platforms live only in stage-page verbs and type-specific fields.

6.1 Module landing / project list

Contents: every project of this platform's type, with stage, owner, client, last activity, and a clear "waiting on you vs. waiting on someone else" indicator; a create-project action. **Accepts:** filters and search; the create-project form (client + type-specific metadata). No file input — this page routes, it does not ingest. **Repository:** reads the project index; writes only a new empty project container.

6.2 Project workspace (the hub of each platform)

Contents: project metadata, current stage, the project's documents shown in visibly separated lanes (human / AI / promoted), a slice of the audit trail, and the stage actions. **Accepts:** file upload, contextually scoped — the attach zone here means "this project, human lane, uploaded by current user," and says exactly that in one line above the drop target. Also metadata edits and stage transitions. **Repository:** reads this project's documents; writes human-lane files only, each stamped with project, stage, actor, timestamp, origin=human.

6.3 Stage / processing page — THE INTEGRITY CHOKEPOINT

Contents: the work surface for the current stage, the in-scope documents, the processing action, the output area. Stage verbs differ by platform (review / analyze / generate) but the structure is identical. **Accepts:** no uploads. Input is *selection from the artifact picker* (Section 7.2), plus processing parameters. **Repository:** reads selected inputs; writes outputs to the AI lane only — automatically, with no human-writable path into the AI lane from this page; records lineage linking inputs to output. **Why this page matters most:** every integrity concern in the product converges here. AI-origin selection is gated (Section 4). AI outputs land in the AI lane automatically and cannot be dropped into the human lane.

6.4 Document detail / provenance page

Contents: the file and its preview; an unmissable origin badge; full lineage (which project/stage created it, by whom or what, which inputs fed it if AI, everywhere it has been reused downstream); version history. **Accepts:** essentially nothing — the only action is promotion, available only on AI-origin artifacts, only to an authorized human, only with a recorded reason. **Repository:** reads file and lineage; on promotion, writes a status change and an audit entry. Origin is never touched.

6.5 Repository browser (the one global, cross-module page)

Contents: search and browse across every project and platform the user is authorized to see; filter by origin lane, project, platform, client, actor, stage, date; origin badges visible on every row. **Accepts:** nothing — no uploads ever happen here. A file with no project context is a file with no provenance. **Repository:** a read-only window.

6.6 Client intake surface

Contents: a stripped, guided render scoped to the single project the client may submit to. States plainly that their files are recorded as client-provided source. No repository browsing, no picker, no visibility into the AI lane. **Accepts:** file upload only, written to the human lane with actor=client and stage=intake. **Repository:** writes only; reads back only confirmation of the client's own submission.

7. Shared components (build once, used everywhere)

7.1 The attach-file component

One component, used on every page that accepts uploads, never reconfigured per page. It is context-aware: it inherits project and lane from wherever it sits and states that inheritance in one short line the user can read before they drop ("Files added here are recorded as human-provided source documents for Project X"). Same visual, same interaction everywhere: drag-and-drop and click-to-browse equally, per-file progress, a clear success state, and on completion a confirmation of exactly where the file landed with a link to it. Two render faces — admin-processing and client-intake — but one destination: the human lane, always, stamped with actor and stage. **Hard rule:** the component physically cannot write to the AI lane. The AI lane is written only by processing steps. If a human has a file, it is by definition human-input. This single constraint removes an entire class of integrity failure and is explainable to an auditor in one sentence.

7.2 The client-scoped, project-linked artifact picker

This is the one place the "parallel but shared" architecture becomes visible to the user, and it is a confirmed build-once component serving all three platforms. It is **not** a free repository search. When an admin starts an engagement for a client, the system already knows the client, so the picker presents recognized, named, dated assets for *that client* — e.g. "This client has a capability list from a Technical Debt project dated [X]. Use it, or upload fresh?" — showing at most that client's existing capability-list versions, not the whole repository. Selecting a cross-project artifact takes a **frozen, version-stamped snapshot at the moment of linking**, never a live reference (Decision #2). AI-origin items in the picker are visually unmistakable and gated by the reuse-moment acknowledgment (Section 4).

7.3 The origin badge / provenance display

A single consistent indicator rendered on every list row, detail header, and picker entry. It must make origin (human / AI / human-authored-AI-informed / promoted) and, in Platform 2, input trust tier, legible at a glance without the user opening anything. This is the visible surface of the entire integrity model; it is not optional anywhere.

8. Per-platform specification

The template (Sections 6-7) is the bulk of every platform. This section specifies purpose, primary users, stages, inputs, outputs, and deviations.

8.1 Platform 1 — Technical Debt / Overlap

Purpose: identify overlapping, redundant, and shadow-IT tooling so the admin can recommend consolidation and redirect wasted spend to gaps and priorities. **Primary user:** admin-heavy. The

client supplies a file and has a conversation; the admin does the work.

Stages:

1. **Raw intake.** Client (or admin) uploads a PDF / Word / Excel (and possibly other formats) listing technology, products, capabilities, software, licenses. Human-lane source.
2. **AI extraction.** The AI parses the messy raw file into a structured capability list. AI-lane output.
3. **Extraction review — its own human checkpoint.** This is the single most important screen in the platform and must not be hidden inside "upload." The admin sees what the AI extracted from the raw file and confirms or corrects it before anything else runs. The admin-confirmed extraction is a distinct artifact (human-authored-AI-informed) sitting between raw upload and overlap analysis — name it explicitly so it is never conflated with the client's original file. Garbage extraction flowing silently into analysis produces wrong recommendations the admin cannot diagnose.
4. **AI overlap analysis.** Identifies duplicates (e.g. multiple SIEM / AV / DLP), shadow IT, and consolidation opportunities with cost implications. AI-lane output.
5. **Conversational interrogation.** The findings are a *workspace*, not a *report*. The admin queries them ("show everything overlapping our SIEM spend," "what would consolidating these three save"). The conversation is AI-lane scratch space. Committing anything from it to the final list is an explicit, deliberate human action — the conversation must never silently leak into the authoritative artifact.
6. **Admin-final reconciled list.** Human-authored, AI-informed. The authoritative capability list for this client.

Outputs / artifacts: the client-level, versioned capability list, in its distinct states (raw client-provided / AI-extracted / admin-confirmed-extraction / admin-final). All states are retained; they must never become visually interchangeable just because they contain similar rows, especially once cost figures are attached.

Deviations from template: the stage page is split across extraction-review and a conversational-findings workspace rather than a single processing surface. Cost data introduces commercial sensitivity — provenance and access discipline here is not just hygiene; the admin-final list drives real budget decisions.

Open behavior to decide later (not blocking): when a client's environment changes, is the list re-uploaded and re-analyzed from scratch, or updated in place? This affects freshness indicators wherever the list is reused downstream.

8.2 Platform 2 — Zero Trust / CSF Compliance Posture

Purpose: assess the client's compliance posture against one chosen framework (CISA ZT, DoD ZT, or CSF), establish desired future posture, and produce a roadmap between them. Distinct

from Platform 3: this is *compliance posture*, not exploitable attack surface. A client can be compliant and exploitable simultaneously; the UI must never let these two questions masquerade as each other.

Primary users: client-heavy (the questionnaire) and reviewer-heavy (the auditable artifacts). The heaviest platform on both the client-facing and audit-facing sides.

Defining first action: framework selection. One framework per engagement. The choice drives which question set the client even sees. It is not buried configuration; it is the engagement's defining act. The same client may run separate engagements per framework over time, all referencing the same client-level capability list.

Stages:

1. **Capability list.** Baseline path is a fresh raw upload. If Platform 1 has been run for this client, the admin-final list is offered as a linkable artifact via the shared picker (Section 7.2), taken as a frozen version-stamped snapshot. No assumption that Platform 1 ran.
2. **Questionnaire + evidence.** A long, structured, high-effort questionnaire completed by the client, with evidence uploads. **One questionnaire, completion-agnostic** — it must work unmodified whether the client is solo or admin-assisted. Do not build two questionnaire UIs; they will drift. Resolve the solo-vs-assisted difference with layered help: a plainly worded question by default, an expandable "what this means / what good evidence looks like" the solo client can open and the assisted session can ignore. Evidence attaches inline at the question it supports, never to a separate pile. Progress is always visible; work is resumable without loss; progressive reveal, not a wall.
3. **Attribution capture.** Every answer carries an attribution tag (Decision #3, resolved — see Section 10). Captured automatically from session context; conservative default (admin present $\Rightarrow$ admin-assisted, never client-asserted); admin can downgrade, never upgrade; per-section override granularity with per-question exception; immutable once submitted; surfaced on every control in the current-state artifact.
4. **AI posture analysis.** AI-lane.
5. **Three distinct linked artifacts (Decision #4)** — never one merged report, because they have different truth-statuses, audiences, and lifespans:
 - **Current-state assessment** — evidence-backed, auditable, factual. Carries the capability-list version stamp (it is only true relative to the inventory it was assessed against). Must keep client-claim / evidence / AI-assessment visibly and structurally separable; a reviewer must see what was claimed, what was provided as proof, and what the AI concluded as three separable things, never one blended narrative.
 - **Desired future-state target** — aspirational stated intent, not evidence-backed. Must never visually read like the current-state assessment.
 - **Transition roadmap** — AI-informed, admin-validated, forward-looking, a living plan. Labeled as a plan, not a finding.

Audit-clarity requirement: the auditor/reviewer is a different reader than the admin or client and needs traceability over polish. The current-state artifact's job is to let an external reviewer walk the line — framework control → client's claim → evidence provided → AI's assessment → identified gap → proposed remediation — for every control, without trusting the tool. Clarity here means defensible, walkable structure, not visual sophistication.

8.3 Platform 3 — MITRE ATT&CK Attack Surface / Gap Analysis

Purpose: map which technologies and capabilities provide detection, prevention, or response to the TTPs and threats in the MITRE ATT&CK matrix, and identify what is not covered — so a CISO understands which threats they cannot detect, prevent, or respond to, and admins/consultants can have the risk conversation about those gaps. Distinct from Platform 2: this is the *actual exploitable attack surface*, not compliance posture.

Primary audience: executive. CISOs, CEOs, CFOs making risk-acceptance decisions — not a technical audience tracing controls.

Stages:

1. **Capability list.** Same as Platform 2 — fresh raw upload as the baseline; Platform 1 admin-final list offered via the shared picker when it exists, frozen snapshot. Independent of Platforms 1 and 2 otherwise.
2. **AI ATT&CK coverage analysis.** Identifies which capability performs which cybersecurity function against which techniques, and produces the uncovered set. AI-lane.
3. **Executive output.** A user-friendly chart/artifact. **Executive reading path first; the ATT&CK matrix is the substrate, not the surface.** A raw matrix handed to a CFO communicates nothing. Lead with the consequence — "these are the attack techniques you are blind to, and here is the risk that carries" — and let the technical matrix be drilled into beneath it. The analysis can be exhaustive; the artifact must not be.

Deviation from template: the output design is inverted relative to Platform 2. Platform 2 needs defensible structure over polish (audit reader). Platform 3 needs executive clarity over technical completeness (decision-maker reader). Same separation principles, opposite presentation priority.

Strategic note (see Section 9): Platform 3 may drive execution of Platform 1's technical-debt analysis — find the gap, fund it from the waste. This is the product's thesis, not an aside.

9. The cross-platform value loop (Decision #5 — flagged, deliberately later phase)

The platforms are independent workflows but compose into one narrative: **the security gaps Platform 3 finds can be funded by the wasted spend Platform 1 finds, with Platform 2's roadmap as the compliance framing.** "Stop paying twice for redundant SIEM; redirect that

money to close the detection gap that leaves you blind to this technique." The whole is worth dramatically more than the parts; this loop is the reason the product is worth building.

Why it is deliberately last. It is simultaneously the highest-value and highest-risk surface in the product. Composing two AI-derived artifacts (Platform 3's gap analysis + Platform 1's overlap analysis) into a third inference that moves a CFO's budget is exactly the moment the integrity model either pays off or fails. The synthesis recommendation must be visibly human-authored-AI-informed, must cite which version of which artifact it drew from, and must never present a composed AI inference as established fact to someone about to act on it.

Design consequence: the architecture already supports this without a fourth platform. The shared client-level spine means the loop needs only a cross-artifact reference capability at the client tier — the natural extension of the picker already being built. The platforms' outputs must be referenceable against each other at the client level (Platform 3's gap citing Platform 1's overlap for the same client; Platform 2's roadmap citing both).

Build rule: build and prove the three platforms individually, on the shared spine, with the integrity model validated, *before* composing them. Composing first, on an unproven base, is precisely how this class of product produces a confident, well-formatted, wrong recommendation that someone acts on.

10. Decision log

The contract. Nothing here was decided by default; each was decided on the merits and is revisitable only explicitly.

Resolved and stable:

- Three parallel, independent platforms over a shared client-level capability list (hub-and-spoke for that one asset; independent otherwise).
- Immutable origin model (human-input / AI-generated / human-authored-AI-informed / promoted); promotion elevates reuse status, never rewrites origin; a file never moves lanes, only copies forward.
- Integrity enforced at the reuse moment (the picker), not at storage.
- One reusable page template instantiated per platform.
- The client-scoped, project-linked artifact picker is a confirmed build-once shared component serving all three platforms.
- Capability list is a versioned, per-client asset living above projects.
- Platform 2 engagement is scoped to one framework, chosen first.
- Platform 2 emits three distinct labeled artifacts (current / desired / roadmap), not one report.
- Platform 3 output is executive-audience-first with the technical matrix as substrate.

#1 — RESOLVED (by Platform 2's actual behavior). Baseline path for every platform is a fresh raw capability-list upload. Platform 1's admin-final list is offered as a linkable artifact only when it exists for that client. No platform assumes another has run.

#2 — RESOLVED (held through Platform 3, no reason to break it). Cross-platform artifact links take a frozen, version-stamped snapshot at the moment of linking, not a live reference. Auditable artifacts must remain accurate about what they were based on.

#3 — RESOLVED. Platform 2 questionnaire answer attribution: per-answer tag, auto-captured from session context, conservative default (admin present $\Rightarrow$ admin-assisted, never client-asserted), admin may downgrade but never upgrade, per-section override granularity with per-question exception, immutable once submitted, surfaced on every control in the current-state artifact as a hard requirement.

#4 — RESOLVED (consistent across platforms). Multi-nature artifacts are separately labeled and individually versioned, never merged into one report. Platform 2's three artifacts and Platform 3's executive-artifact-plus-technical-substrate both follow this convention.

#5 — FLAGGED, NOT YET BUILT. The cross-platform value loop is a deliberate later phase. Build and prove the three platforms individually on the shared spine with the integrity model validated first. Highest value, highest risk; must not be built first or by default.

11. Deliberately out of scope for this document

Stated honestly so nothing is assumed resolved that is not:

- Screen-level visual layouts and the visual design system (the next phase after this spec; a diagram orientation layer is the planned follow-up).
 - The actual question sets for Platform 2's three frameworks.
 - The AI engine internals (extraction, overlap, posture, ATT&CK coverage logic).
 - The database schema and the JSON/Python consolidation approach (developer's implementation).
 - Platform 1's re-upload-vs-update behavior on client environment change (flagged in 8.1; affects downstream freshness indicators; decide before Platform 1 build).
 - Role/permission model specifics beyond the client/admin/reviewer distinctions implied here.
-

12. Recommended build sequence

1. The shared spine: identity, the client tier, the repository with the lane/origin model, the

audit log. Nothing renders trustworthy without this.

2. The shared components: the attach component, the artifact picker, the origin badge. These are reused by everything; building them first forces the integrity model to be real before any screen depends on it.
3. The shared page template, instantiated bare.
4. Platform 1 end-to-end (it produces the shared asset the others consume; proving it first de-risks the others).
5. Platform 3, then Platform 2 (Platform 2 is the heaviest; sequencing it after a second platform proves the template flexes across very different primary users).
6. Only then, Decision #5: the cross-platform value loop, as a deliberate phase with the integrity model already validated in production on the individual platforms.

The integrity model is the non-negotiable spine of this handoff, not an appendix to it. If it is treated as "combine the backends and wire up the screens," the integrity model is the thing that quietly disappears under schedule pressure — and it is the one part whose absence turns a credible product into one that confidently hands a decision-maker a wrong number.